

Notebook 07 — Variant of Greedy, explicado fácil

El problema, en palabras

Imagina un salón con varios niños. Algunos podrían estar resfriados, pero no sabemos cuáles. En vez de revisar a cada niño por separado, hacemos una **prueba de grupo**: juntamos a varios niños y la prueba dice si el grupo está limpio.

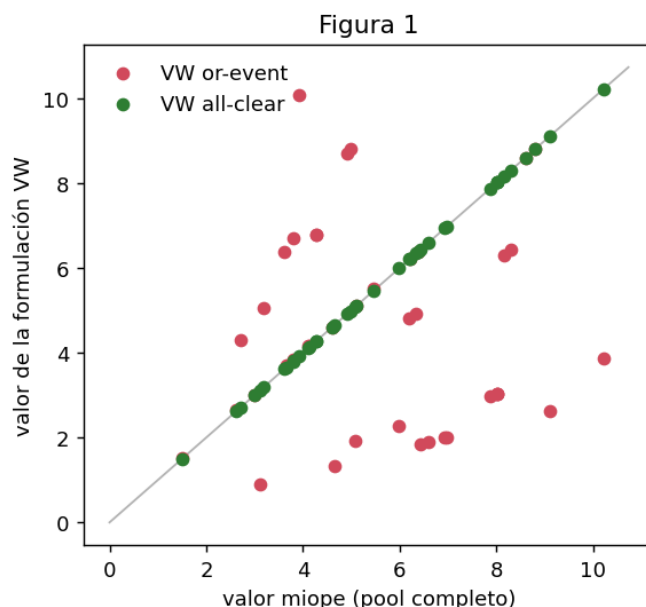
- Si el grupo sale **limpio**, ¡sabemos de un solo golpe que *todos* esos niños están sanos!
- Solo podemos hacer **pocas pruebas** (las llamamos B).
- Cada niño tiene un **valor**: qué tan importante es darle de alta pronto. Queremos dar de alta a los niños más valiosos posible.

El **algoritmo greedy** (avaricioso) es la estrategia más sencilla: en cada paso elige el grupo que se ve mejor *en ese momento*.

La idea de los “paquetes” (super-nodos). Después de unas cuantas pruebas, algunos niños ya fueron “tocados”. La idea de Marmolejo es: en vez de pensar niño por niño, podemos juntar a varios niños ya tocados y tratarlos como un solo **paquete**. A esto le llamamos formulación VW.

Nota: para la probabilidad de que un grupo salga limpio multiplicamos la de cada niño como si fueran independientes. Es una simplificación; tras una prueba con un positivo los niños quedan algo relacionados y el cálculo exacto puede cambiar un poco.

Figura 1 — partir un grupo en dos no cambia su puntaje



Analogía. Vas al súper con una canasta. Para saber cuánto cuesta, puedes sumar todos los productos juntos, *o* separar la canasta en “frutas” y “verduras”, sacar el precio de cada parte y juntarlos. Da *el mismo total*: partir la canasta no cambia el precio. La formulación VW hace justo eso con un grupo de niños: lo parte en dos pedazos y los vuelve a juntar.

A cada grupo le damos un **puntaje**. La fórmula es sencilla:

$$\text{puntaje} = \underbrace{(v_1 + v_2 + \dots)}_{\text{valor total del grupo}} \times \underbrace{(c_1 \times c_2 \times \dots)}_{\text{probabilidad de salir limpio}}$$

donde v_i es el valor del niño i y c_i es su probabilidad de estar sano.

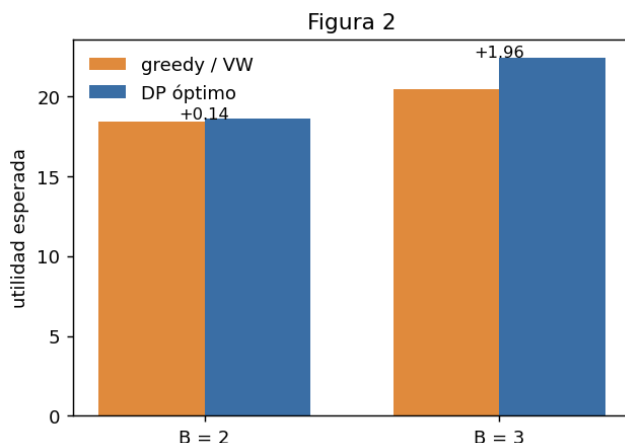
Ahora partimos el grupo en dos pedazos: T (niños ya tocados) y U (niños nuevos). Pasan dos cosas:

- **Sumar por pedazos da lo mismo:** $v_1 + v_2 + \dots = (\text{suma de } T) + (\text{suma de } U)$.
- **Multiplicar por pedazos da lo mismo:** $c_1 \times c_2 \times \dots = (\text{producto de } T) \times (\text{producto de } U)$.

Por eso el puntaje sale igual lo calcules de un tirón o por pedazos. En la figura, cada punto es un grupo posible: el eje horizontal es su puntaje calculado “de un tirón” y el vertical es el puntaje calculado “por pedazos” (VW). Los puntos **verdes** caen *exactamente sobre la línea diagonal*: misma cuenta, mismo resultado. Los puntos **rojos** usan otra regla para combinar las probabilidades y por eso se salen de la línea.

Pregunta para Francisco. Que la formulación VW dé exactamente el mismo puntaje que el greedy de siempre: ¿es la noticia que queríamos (VW es tan fácil de manejar como el greedy) o más bien quiere decir que todavía no estamos ganando nada nuevo?

Figura 2 — pensar solo en el siguiente paso cuesta caro



Analogía. En el ajedrez, un jugador *miope* hace la jugada que se ve mejor ahora mismo. Un jugador que *planea* piensa varias jugadas hacia adelante. Si solo queda una jugada, planear casi no ayuda. Pero si quedan tres, el que planea le gana claramente al miope.

El greedy (y por la Figura 1, también VW) es **miope**: solo mira el siguiente paso. La mejor estrategia posible, que planea hasta el final, se calcula con *programación dinámica* (DP). La figura compara las dos:

$$\text{brecha} = (\text{utilidad del que planea todo}) - (\text{utilidad del miope})$$

- Con $B = 2$ pruebas, la brecha es chiquita: $\approx 0,14$.
- Con $B = 3$ pruebas, la brecha es mucho mayor: $\approx 1,96$.

Conclusión: **mientras más pruebas queden por hacer, más caro sale ser miope**. La formulación VW, por sí sola, no cierra esa brecha.

Pregunta para Francisco. La brecha crece de $B = 2$ a $B = 3$. ¿Esperarías que se estabilice a partir de cierto número de pruebas, o que siga creciendo mientras queden pruebas por gastar?

Figuras 3 y 4 — las primeras pruebas de RL (en breve)

Figura 3 — una computadora que aprende jugando. En vez de darle la estrategia, dejamos que una computadora juegue muchísimas partidas y aprenda sola (esto es *reinforcement learning*). La figura muestra que, tras muchas partidas, juega casi tan bien como la mejor estrategia posible (el DP). Es una forma de comprobar que el planteamiento es correcto.

Pregunta para Francisco. La computadora se acerca al óptimo pero, con un número limitado de partidas, se queda un poquito por debajo. ¿Te interesa más una garantía de que llega exacto, o saber cuántas partidas bastan para quedar muy cerca?

Figura 4 — el método exacto se vuelve imposible. Calcular la estrategia perfecta es como un rompecabezas que se duplica de tamaño cada vez que entra un niño nuevo. La figura mide el tiempo y muestra que, pasados unos 14 niños, ya no se puede en la práctica. Por eso hace falta aprender (RL) o simplificar (poda con VW).

Pregunta para Francisco. *El cálculo exacto se “estrella” alrededor de $n = 14$. Antes de saltar a métodos de RL más pesados, ¿vale la pena formalizar una poda con VW que achique el problema, o usamos esta pared como motivación directa para RL?*